

GtelPay Core — Design Decision Brief

A fiat accounting + wallet core (core.accounting + core.wallet). One page for decision-makers; the deep specs are backing, not shown here.

The problem

In payments, correctness **is** money. The failure modes are not cosmetic:

- **Overspend / double-spend** — a member spends funds twice (retry, race, timeout).
- **Unbalanced or editable books** — the ledger can't be trusted for audit or regulators.
- **Speed vs truth conflict** — “how much can this member spend *now*?” needs millisecond reads; “what happened in the ledger?” needs immutable, auditable accounting. Forced into one model, one of them breaks.

A generic CRUD wallet table handles none of these safely. We need a core built for money.

The approach (three ideas)

1. **Two domains, split on purpose.** `core.wallet` = fast spendable balance (hot path). `core.accounting` = immutable double-entry ledger (system of record). Opposite consistency needs → kept separate, never sharing storage.
2. **Correctness by construction.** Every balance change is **idempotent** and **recorded in the same transaction** (no balance moves without a movement row). The ledger is always **balanced** (debits = credits) and **never edited** — corrections are reversing entries. Transit accounts net to zero on every completed flow.
3. **Orchestration, not distributed transactions.** A thin sequencer coordinates the two domains via **saga + compensation** — no fragile 2-phase commit.

“Isn't this over-engineering?” — No. Here's the test.

Over-engineering = complexity with **no payoff**. Every piece here maps to a **specific, expensive money-loss failure**. Remove it and you don't get “simpler” — you get a named risk.

Component	The failure it prevents	Cost if removed
Idempotency (under lock)	Retry / race → double-spend, double-credit	Direct cash loss, every retry storm
Double-entry + immutable ledger	Unbalanced or edited books	Fails audit / regulator ; can't prove balances
Wallet ⊥ accounting split	One model can't be both fast <i>and</i> immutable-auditable	Either slow spend checks or an untrustworthy ledger
Transit net-zero	Money “stuck” mid-flow, untracked	Silent imbalance, reconciliation breaks

Component	The failure it prevents	Cost if removed
Saga + compensation	Partial cross-domain failure leaves funds inconsistent	Stranded/duplicated money on any outage

The honest question isn't "is this complex?" — it's "which safeguard do you want to cut, and which loss do you accept?" Each one is table-stakes for a regulated money system (the same primitives banks, Stripe, Modern Treasury all use).

What we deliberately did not build (proof of restraint)

Real over-engineering is gold-plating. We refused it on the record (ADRs):

- **No microservice sprawl** — 4 modules, not 40. One shared lib, kept minimal (ADR-002 explicitly rejects a second "common" module).
- **No 2-phase commit, no Temporal** — plain RabbitMQ workers (ADR-035); saga over the network seam, nothing heavier.
- **No custom ledger database** — plain PostgreSQL, two schemas (ADR-003).
- **No event-sourcing, no CQRS framework** — a snapshot balance + an append-only log, that's it (ADR-004).
- **Single currency (VND) v1, features opt-in** — scope held deliberately small (ADR-019); nothing built "just in case."

A truly over-engineered design wouldn't have a "what we refused" list this long.

Why it's credible — this is not paper, it runs

- `core.foundation`, `core.wallet`, `core.accounting`, `app-orchestration` are **implemented (Java / Spring Boot 3) with passing tests** — not slideware.
- The money-critical safety rules are **coded and tested**: authoritative single-row balance + append-only movement in one DB transaction; **idempotency re-checked under lock** (a retry can't double-apply); balanced, immutable journals; two-phase deposit (funds credited only after the ledger posts).
- **41 Architecture Decision Records** lock each choice with acceptance criteria + test cases; a **SQL invariant suite gates the build** (e.g. every journal balances, transit nets to zero) — so drift fails CI instead of reaching production.

Build vs. buy

We evaluated off-the-shelf ledgers (Blink, Formance, TigerBeetle). They **merge balance and ledger into one model** — excellent for an embedded ledger primitive. Our requirements (a real chart of accounts, transit net-zero, end-of-day settlement, regulator-grade audit trail, and a separate high-RPS spendable balance) need the **explicit split**. Decision: **borrow their proven patterns, build the parts that must fit bank/audit reality**. The build is small, focused, and already running.

Risk is controlled

- **Backward compatible** — new capability is opt-in; existing callers are unaffected.

- **Reconciliation is report-only** — it surfaces drift, never silently rewrites the books.
- **Contracts (OpenAPI / AsyncAPI) are the source of truth** — wire shape can't drift from intent.

Scope

`core.accounting` (ledger) + `core.wallet` (member balance) + `core.foundation` (shared lib) + `app-orchestration` (sequencer). **Fiat (VND)**. Full ADRs, process flows, contracts, and 150+ acceptance scenarios exist as backing material, available on request.

Backing: adr/ (41 ADRs + AC/TC) · spec/ (foundation, processes, contracts) · design-v2/acceptance.md (acceptance scenarios) · platform/ (running modules + tests).